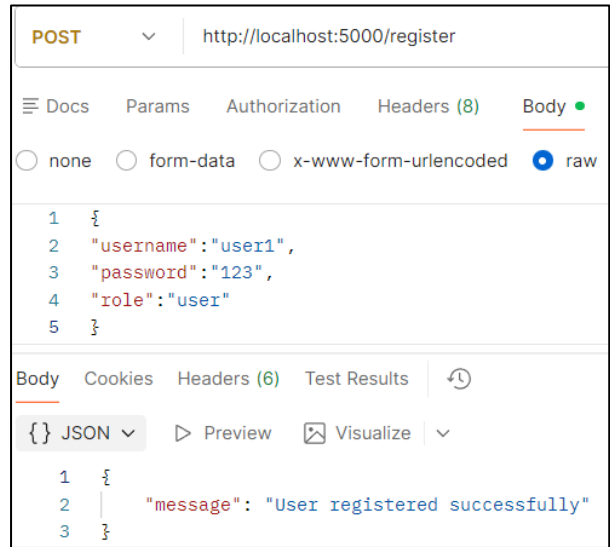
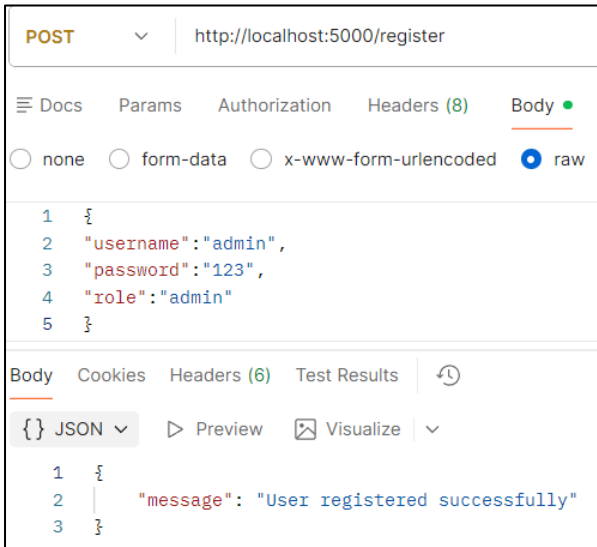


Practical No. 8

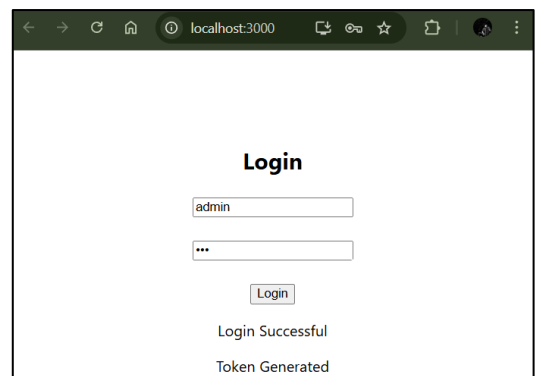
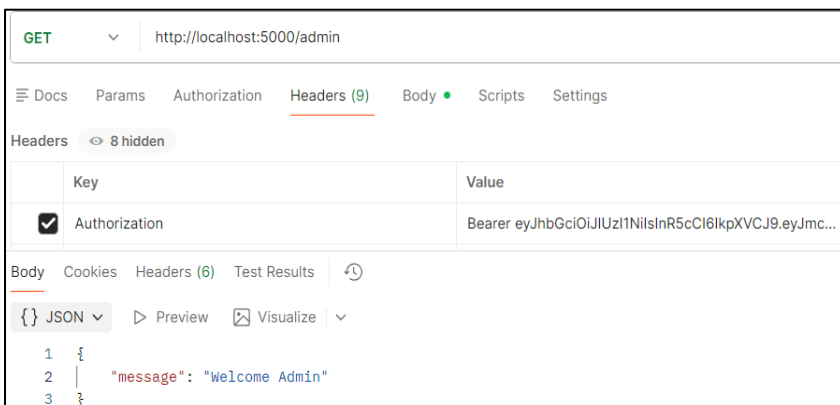
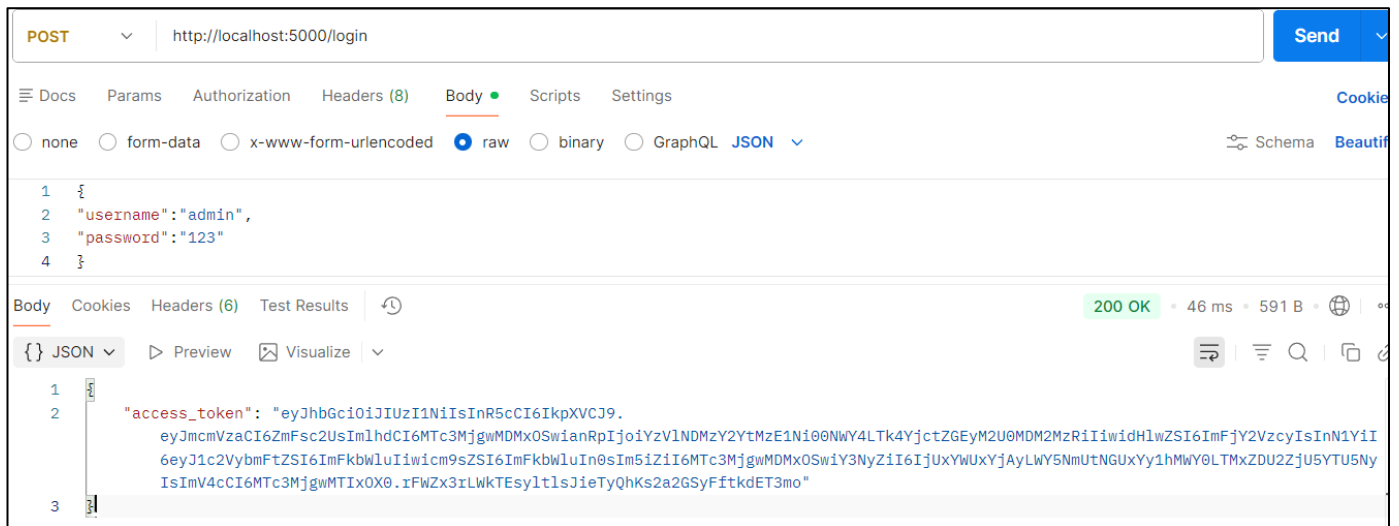
Aim: To deploy a Full Stack Web Application on a cloud platform and implement user authentication with role-based access control (RBAC) using backend APIs.

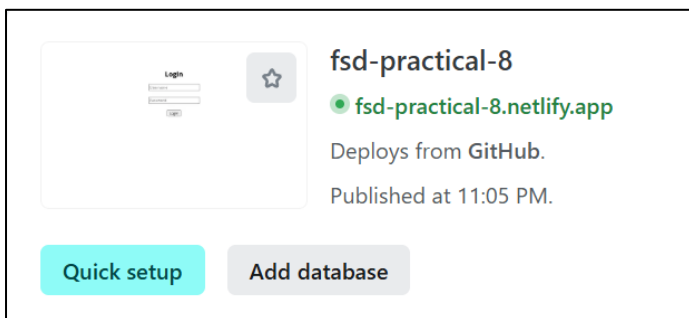
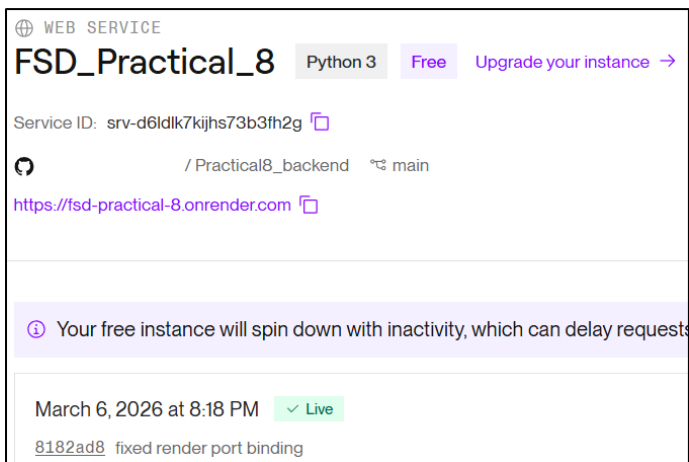
Output:



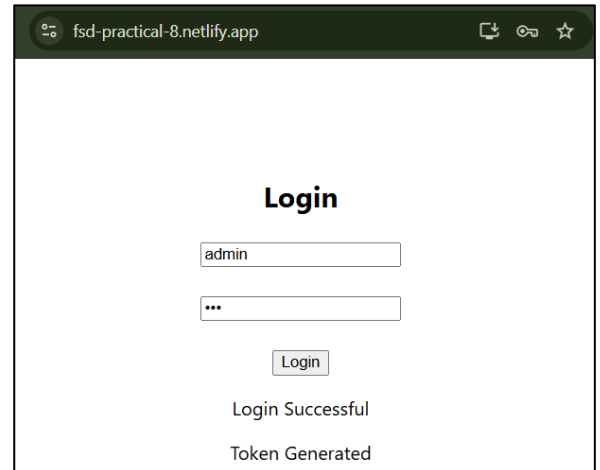
The screenshot shows a database table named 'users' with the following data:

id	username	password	role
1	admin	123	admin
2	user1	123	user





Backend Deployed on Render



Frontend Deployed on Netlify

Code:

app.py:

```
from flask import Flask, request, jsonify
from flask_jwt_extended import (
    JWTManager,
    create_access_token,
    jwt_required,
    get_jwt_identity,
    get_jwt
)
from flask_cors import CORS
import sqlite3
app = Flask(__name__)
CORS(app)
# Secret key for JWT
app.config["JWT_SECRET_KEY"] = "secretkey123"
jwt = JWTManager(app)
# Database connection
def get_db():
    conn = sqlite3.connect("users.db")
    conn.row_factory = sqlite3.Row
    return conn
# Create table if it doesn't exist
conn = get_db()
conn.execute("""
CREATE TABLE IF NOT EXISTS users (
```

```
id INTEGER PRIMARY KEY AUTOINCREMENT,
username TEXT,
password TEXT,
role TEXT
)
""")
conn.commit()
conn.close()
# Home route
@app.route("/")
def home():
    return "Backend Running Successfully"
# Register user
@app.route("/register", methods=["POST"])
def register():
    data = request.json
    conn = get_db()
    conn.execute(
        "INSERT INTO users (username, password, role)
VALUES (?, ?, ?)",
        (data["username"], data["password"],
data["role"])
    )
    conn.commit()
```

```

    conn.close()
    return jsonify({"message": "User registered
successfully"})
# Login route
@app.route("/login", methods=["POST"])
def login():
    data = request.json
    conn = get_db()
    user = conn.execute(
        "SELECT * FROM users WHERE username=?
AND password=?",
        (data["username"], data["password"])
    ).fetchone()
    conn.close()
    if user:
        token = create_access_token(
            identity=user["username"], # identity must
be string
            additional_claims={"role": user["role"]} #
store role here
        )
        return jsonify(access_token=token)
    return jsonify({"message": "Invalid credentials"}),
401
# Admin route (role protected)
@app.route("/admin", methods=["GET"])
@jwt_required()
def admin():
    claims = get_jwt()
    if claims["role"] != "admin":
        return jsonify({"message": "Access denied"}),
403
    return jsonify({"message": "Welcome Admin"})
# Normal user route
@app.route("/user", methods=["GET"])
@jwt_required()
def user():
    username = get_jwt_identity()
    return jsonify({"message": f"Welcome
{username}"})
# Run server
import os
if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port)

```

App.js:

```

import React, { useState } from "react";
import axios from "axios";
function App() {
    const [username, setUsername] = useState("")
    const [password, setPassword] = useState("")
    const [token, setToken] = useState("")
    const [message, setMessage] = useState("")
    const login = async () => {
        try{
            const res = await axios.post("https://fsd-
practical-8.onrender.com/login",{
                username,
                password
            })
            setToken(res.data.access_token)
            setMessage("Login Successful")
        }
        catch(err){
            setMessage("Invalid Login")
        }
    }
    return (
        <div
style={{textAlign:"center",marginTop:"100px"}}>
            <h2>Login</h2>
            <input
placeholder="Username"
onChange={(e)=>setUsername(e.target.value)}
/>
            <br/><br/>
            <input
type="password"
placeholder="Password"
onChange={(e)=>setPassword(e.target.value)}
/>
            <br/><br/>
            <button onClick={login}>Login</button>
            <p>{message}</p>
            {token && <p>Token Generated</p>}
        </div>
    )
}
export default App

```